

Generic POSIX IPC Lab Guide

for `posix_ipc_helpers.c`

Practical reference for shared memory, mmap, semaphores, and sync primitives

April 15, 2026

Purpose of this document

This guide explains how to use the generic helper file `posix_ipc_helpers.c` in future Operating Systems labs where the story changes, but the technical requirements are similar:

- shared memory via `shm_open` and `mmap`,
- anonymous shared mappings for control structures,
- named semaphores for capacity/resource limits,
- process-shared `pthread` synchronization,
- robust mutex handling when a process crashes while holding a lock.

1. Mental model for typical lab tasks

Most tasks can be split into two memory areas:

1. **Control area** in anonymous shared memory (`MAP_SHARED|MAP_ANONYMOUS`):
barrier, mutexes, flags, condition variables.
2. **Data area** in named shared memory object (`shm_open + mmap`):
arrays/structs representing the shared state.

Named semaphores then regulate capacity or resource access (e.g., “max users per resource”).

2. Quick start integration

2.1. Build command

```
1 gcc -std=c11 -Wall -Wextra -O2 your_solution.c posix_ipc_helpers.c -o  
   your_solution -pthread -lrt
```

2.2. Minimal include setup in your main solution file

```
1 #include <pthread.h>  
2 #include <semaphore.h>  
3 #include <sys/mman.h>  
4 #include <sys/wait.h>  
5 #include <unistd.h>  
6  
7 /* Add prototypes manually or create a header later */
```

3. Helper groups and when to use them

3.1. Utility helpers

- `ipch_msleep(ms)`: sleep in milliseconds with EINTR-safe retry.
- `ipch_seed_rng()`: process-specific random seed.
- `ipch_rand_inclusive(lo, hi)`: convenient random range.
- `ipch_flat_index(row, col, cols)`: map 2D coordinates to 1D array index.

3.2. Named semaphore helpers

- `ipch_sem_open_named(name, init)`.
- `ipch_sem_wait_intr(sem)` and `ipch_sem_post_checked(sem)`.
- `ipch_sem_close_checked(sem)` and `ipch_sem_unlink_if_exists(name)`.
- Batch variants for indexed semaphores: `ipch_open_sem_array`, `ipch_close_sem_array`, `ipch_unlink_sem_array_if_exists`.

Use these when:

- each resource has capacity > 1 ,
- you need global named objects visible across processes,
- you need easy cleanup between reruns.

3.3. Anonymous shared memory helpers

- `ipch_mmap_anon_shared(bytes)`.
- `ipch_munmap_checked(ptr, bytes)`.

Use these when:

- parent and children need shared control metadata only,
- no global name is required in the filesystem namespace.

3.4. Named shared memory helpers

- `ipch_shm_open_create_rw`, `ipch_shm_open_rw`.
- `ipch_ftruncate_checked`, `ipch_mmap_shared_rw`.
- One-stop creator: `ipch_create_map_shm_rw`.
- cleanup: `ipch_close_fd_checked`, `ipch_shm_unlink_if_exists`.

Use these when:

- shared data must survive beyond single mapping operation lifetime,
- multiple independent processes may open the same named data block.

3.5. Process-shared pthread primitive helpers

- Barrier: `ipch_barrier_init_pshared, ipch_barrier_wait_checked`.
- Mutex attrs: `ipch_mutexattr_init_pshared(attr, robust)`.
- Robust recovery: `ipch_mutex_lock_robust_recover`.
- Condvars (optional): `ipch_condattr_init_pshared, ipch_cond_*`.

4. Canonical pattern for lab solutions

4.1. Step 1: pre-clean named resources

```
1 ipch_unlink_sem_array_if_exists(resource_count, "/lab-sem-");
2 ipch_shm_unlink_if_exists("/lab-shm");
```

This avoids stale state from previous crashes/runs.

4.2. Step 2: create shared control block

```
1 typedef struct {
2     pthread_barrier_t start_barrier;
3     pthread_mutex_t state_mutex;
4     int stop_flag;
5 } ctrl_t;
6
7 ctrl_t *ctrl = ipch_mmap_anon_shared(sizeof(*ctrl));
8 ipch_barrier_init_pshared(&ctrl->start_barrier, n_children + 1);
9
10 pthread_mutexattr_t ma;
11 ipch_mutexattr_init_pshared(&ma, true); /* robust=true optional */
12 ipch_mutex_init_checked(&ctrl->state_mutex, &ma);
13 pthread_mutexattr_destroy(&ma);
```

4.3. Step 3: fork children

```
1 ipch_spawn_children(n_children, child_entry, child_arg);
```

4.4. Step 4: create named shared data block (parent)

```
1 size_t count = rows * cols;
2 size_t bytes = count * sizeof(double);
3 int shm_fd;
4 double *data = ipch_create_map_shm_rw("/lab-shm", bytes, &shm_fd);
5 for (size_t i = 0; i < count; ++i) data[i] = 1.0;
```

4.5. Step 5: release barrier once parent setup is ready

```
1 ipch_msleep(500); /* if required by assignment */
2 ipch_barrier_wait_checked(&ctrl->start_barrier);
```

4.6. Step 6: children map shared data and work

```
1 /* child side */
2 ipch_barrier_wait_checked(&ctrl->start_barrier);
3
4 int cfd = ipch_shm_open_rw("/lab-shm");
5 double *arr = ipch_mmap_shared_rw(cfd, bytes);
6
7 sem_t *gate = ipch_sem_open_named("/lab-sem-0", CAP);
8 ipch_sem_wait_intr(gate);
9 /* critical update */
10 ipch_sem_post_checked(gate);
```

4.7. Step 7: cleanup

```
1 ipch_wait_all_children();
2 ipch_munmap_checked(data, bytes);
3 ipch_close_fd_checked(shm_fd);
4 ipch_shm_unlink_if_exists("/lab-shm");
5 ipch_unlink_sem_array_if_exists(resource_count, "/lab-sem-");
```

5. Robust mutex crash-recovery workflow

When robust mode is enabled and a process dies while holding a mutex:

- next locker gets EOWNERDEAD,
- state may be inconsistent and must be repaired,
- `pthread_mutex_consistent()` must be called before unlock.

In the helper:

```
1 int rc = ipch_mutex_lock_robust_recover(&ctrl->state_mutex);
2 if (rc == 1) {
3     /* previous owner died; repair shared state here */
4 }
5 /* use protected state */
6 ipch_mutex_unlock_checked(&ctrl->state_mutex);
```

6. API correctness notes

- `sem_unlink` and `shm_unlink` remove names, not active mappings/handles.
- `ftruncate` must be called before mapping a newly created shared-memory object to desired size.
- `MAP_SHARED` is required for inter-process visibility of writes.
- Process-shared pthread objects must live in shared memory.
- Barrier count must include every process/thread that will wait on it.
- For `pthread_*` functions returning error codes directly, helper converts them into consistent fatal handling.

7. Common exam mistakes and quick fixes

- **Mistake:** Forgot leading slash in names.
Fix: Always use names like `"/lab-shm"`, `"/lab-sem-0"`.
- **Mistake:** Child uses shared memory before parent initialized data.
Fix: Add startup barrier and release only after initialization.
- **Mistake:** Updated shared array without lock/semaphore around read-modify-write.
Fix: Wrap update in correct synchronization primitive.
- **Mistake:** Used robust mutex but ignored `EOWNERDEAD`.
Fix: recover path + `pthread_mutex_consistent`.
- **Mistake:** Re-run fails because named objects still exist.
Fix: pre-clean with `unlink-if-exists` helpers.

8. Suggested adaptation strategy on unknown tasks

1. Identify resource model (single array? many resources? queue?).
2. Pick synchronization:
 - semaphore for capacity counters,
 - mutex for mutual exclusion,
 - barrier for start ordering,
 - condvar for wait-notify workflow.
3. Build minimal working pipeline first (fork, map, one update).
4. Add assignment-specific behavior (random events, termination flags, outputs).
5. Add full cleanup last and test rerun.

Final tip

Treat `posix_ipc_helpers.c` as a toolbox, not a final solution template. On lab day, copy only the functions you need, keep names consistent, and focus on synchronization invariants first.